LangChain v1.2.x Quick Reference

Condensed Cheat Sheet ▪ March 2026

A quick-lookup reference for the 20 most essential LangChain patterns

Essential Imports (Top 20)

Models & Chat

```
from langchain_openai import
    ChatOpenAI
from langchain_anthropic import
    ChatAnthropic
from langchain_google_vertexai import
    ChatVertexAI
from langchain_ollama import
    ChatOllama
```

Prompts

```
from langchain_core.prompts import (
  ChatPromptTemplate,
  MessagesPlaceholder,
  PromptTemplate
)
```

Chains & Tools

```
from langchain_core.tools import tool
from langchain.agents import (
  create_react_agent,
  AgentExecutor
)
```

RAG & Memory

```
from langchain_text_splitters import (
  RecursiveCharacterTextSplitter
)
from langchain_core.runnables import (
  RunnableWithMessageHistory,
  RunnablePassthrough
)
from langchain_chroma import Chroma
```

Essential Imports (Continued)

Outputs & Parsing

```
from langchain_core.output_parsers
    import (
  JsonOutputParser,
  StrOutputParser,
  PydanticOutputParser
)
from langchain.callbacks.tracers.
    langchain import (
  LangChainTracer
)
```

LangGraph

```
from langgraph.graph import StateGraph
from langgraph.prebuilt import (
  create_react_agent
)
```

Documents & Loaders

```
from langchain_community.
    document_loaders import (
  PyPDFLoader,
  WebBaseLoader,
  CSVLoader
)
from langchain_core.documents import
    Document
```

Chat Models — Quick Init

```python
# OpenAI
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0, api_key="...")

# Anthropic
from langchain_anthropic import ChatAnthropic
llm = ChatAnthropic(model="claude-3-5-sonnet-20241022", temperature=0)

# Google VertexAI
from langchain_google_vertexai import ChatVertexAI
llm = ChatVertexAI(model="gemini-2.0-flash", temperature=0)

# Ollama (Local)
from langchain_ollama import ChatOllama
llm = ChatOllama(model="llama2", base_url="http://localhost:11434")

# Universal invoke pattern
response = llm.invoke("Your prompt here")
result = llm.batch(["prompt1", "prompt2"])  # Parallel
async_result = await llm.ainvoke("prompt")   # Async
```

[Docs: langchain-ai/langchain]

Prompt Templates — 3 Patterns

```python
# 1. Simple string template
from langchain_core.prompts import PromptTemplate
prompt = PromptTemplate.from_template("Explain {topic}")
prompt.invoke({"topic": "neural nets"})

# 2. Chat format (preferred for models)
from langchain_core.prompts import ChatPromptTemplate
chat_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a {role}"),
    ("human", "{question}"),
    MessagesPlaceholder(variable_name="history")
])

# 3. from_template shortcut
chat_prompt = ChatPromptTemplate.from_template(
    "System: {system}\nUser: {user}"
)
```

Key: Use `ChatPromptTemplate` for chat models, nest `MessagesPlaceholder` for history.

[Docs: Prompts]

LCEL Patterns (The 5 Most Common)

```python
# 1. Basic chain: prompt -> llm -> parser
chain = prompt | llm | StrOutputParser()
result = chain.invoke({"key": "value"})

# 2. Parallel branches (map)
from langchain_core.runnables import RunnableParallel
parallel = RunnableParallel(
    analysis=prompt | llm,
    summary=other_prompt | llm
)

# 3. Passthrough (preserve input)
from langchain_core.runnables import RunnablePassthrough
chain = ({"doc": RunnablePassthrough()} | llm)

# 4. Branching (conditional)
chain = condition | (true_branch | llm) or (false_branch | llm)

# 5. With fallbacks
chain = (prompt | llm).with_fallbacks([fallback_chain])
```

[Docs: LCEL]

Tools & Agents — Minimal

```python
# Define tool with decorator
from langchain_core.tools import tool

@tool
def multiply(a: int, b: int) -> int:
    """Multiply two numbers."""
    return a * b

# Bind tools to model
from langchain_core.tools import bind_tools
model_with_tools = llm.bind_tools([multiply])

# Create agent
from langchain.agents import create_react_agent, AgentExecutor
from langchain import hub
prompt = hub.pull("hwchase17/react")
agent = create_react_agent(llm, [multiply], prompt)
executor = AgentExecutor(agent=agent, tools=[multiply])

result = executor.invoke({"input": "What is 5 times 3?"})
```

[Docs: Agents]

RAG Pipeline — Complete Minimal Setup

```python
# 1. Load documents
from langchain_community.document_loaders import WebBaseLoader
loader = WebBaseLoader("https://example.com")
docs = loader.load()

# 2. Split
from langchain_text_splitters import RecursiveCharacterTextSplitter
splitter = RecursiveCharacterTextSplitter(chunk_size=1000)
splits = splitter.split_documents(docs)

# 3. Embed & store
from langchain_openai import OpenAIEmbeddings
from langchain_chroma import Chroma
embeddings = OpenAIEmbeddings()
vectorstore = Chroma.from_documents(splits, embeddings)

# 4. Retrieve & generate
retriever = vectorstore.as_retriever()
chain = {"context": retriever, "question": RunnablePassthrough()} \
    | prompt | llm | StrOutputParser()
answer = chain.invoke("What does the site say about X?")
```

[Docs: RAG]

Memory Patterns

```python
# 1. RunnableWithMessageHistory (stateful chains)
from langchain_core.runnables.history import (
  RunnableWithMessageHistory
)
chain = RunnableWithMessageHistory(
    runnable=llm,
    get_session_history=get_session_history,
    input_messages_key="input",
    history_messages_key="history"
)
# Invoke with session_id
result = chain.invoke(
    {"input": "Hi!"},
    config={"configurable": {"session_id": "user123"}}
)

# 2. trim_messages (summarize old context)
from langchain_core.messages import trim_messages
trimmer = trim_messages(
    max_tokens=1000,
    strategy="last"
)

# 3. Store in any backend (Redis, PostgreSQL, etc.)
```

LangGraph — State Machines

```python
from langgraph.graph import StateGraph
from typing import TypedDict

class AgentState(TypedDict):
    messages: list
    counter: int

# Build graph
graph = StateGraph(AgentState)

# Add nodes (functions)
def node_1(state):
    return {"messages": state["messages"] + ["step1"], "counter": 1}

def node_2(state):
    return {"messages": state["messages"] + ["step2"]}

graph.add_node("node1", node_1)
graph.add_node("node2", node_2)

# Add edges
graph.add_edge("node1", "node2")
graph.set_entry_point("node1")
graph.set_finish_point("node2")
```

Streaming & Callbacks

```python
# Stream tokens
for chunk in llm.stream("Hello"):
    print(chunk.content, end="", flush=True)

# Async streaming
async for chunk in await llm.astream("Prompt"):
    print(chunk.content)

# Stream with events
from langchain_core.callbacks import LangChainTracer
from langsmith import Client
client = Client()
tracer = LangChainTracer(project_name="my_project")

result = chain.invoke(
    {"input": "..."},
    config={"callbacks": [tracer]}
)

# View in LangSmith
# https://smith.langchain.com -> projects -> my_project

# Custom callback
from langchain_core.callbacks import BaseCallbackHandler
```

Top 10 Errors & Fixes

portError → Wrong module. Use `langchain_openai`, not `langchain.llms`

y missing → Set env var: `export OPENAI_API_KEY=...` or pass to constructor

voke fails → Input dict keys must match prompt placeholders

rns None → Check `temperature`, `max_tokens`, provider limits

rser error → Output format != parser expectation. Test LLM output directly.

ding fails → Tools must be List[BaseTool]. Use `@tool` decorator.

ns empty → Vectorstore empty or wrong embedding model

persisting → Check `session_id` in config, backend connectivity

ntax error → Ensure left side outputs right side's expected input type

tateError → State keys must match TypedDict definition exactly

Pro tip: Always test components individually before chaining them.

Decision Matrix: What to Use When

Scenario	Chain	Agent	LangGraph	Deep Agent
Fixed workflow	X			
Dynamic tool use		X	X	
Multi-step reasoning		X	X	X
Parallel tasks	X		X	X
Complex state mgmt			X	X
Human-in-loop			X	X
Real-time streaming	X	X	X	
Simple API call	X			
RAG retrieval	X	X	X	
Agentic loops		X	X	X

Rule of Thumb

Start with **Chain**, add **Agent** for tool use, use **LangGraph** for fine-grained control.

Performance & Best Practices

Optimization

- Use `.batch()` for parallel inference (faster than loop)
- Stream tokens with `.stream()` for UX (no waiting for full response)
- Cache embeddings—don't re-embed the same docs
- Use cheaper models for routing/filtering, expensive for final answer
- Set `temperature=0` for deterministic outputs (summaries, extraction)
- Chunk size 500–1500 tokens is sweet spot (not too small, not too large)

Common Pitfalls

- Don't reinitialize embeddings for each query (slow, inconsistent)
- Don't use retriever without persistence check (data loss)
- Don't forget to add error handling in agent loops (infinite loops)

Resources & Next Steps

Official Docs

- `https://python.langchain.com`
- LangGraph: `https://langchain-ai.github.io/langgraph`
- LangSmith: `https://smith.langchain.com`

Key Concepts

- LCEL (Runnable chains)
- Output parsers
- Tool schemas
- State graphs

Common Extensions

- `langchain-openai`
- `langchain-anthropic`
- `langchain-community`
- `langgraph`

Next: Deep Dive Into

- Fine-tuning with function calling
- Multi-agent coordination
- Custom memory backends
- Production deployment

This is a quick reference—bookmark it!